MEMORIA TECNICA

QuoteMatic

Aplicacion backend con minifront EJS para recomendacion de frases

Proyecto individual - Bootcamp Full Stack

Campo	Detalle
Autor	David Lopez Sotelo
Repositorio	github.com/David-LS-Bilbao/QuoteMatic
Rama de referencia	dev
Stack principal	Node.js, Express, TypeScript, MongoDB, Mongoose, EJS
Fecha del borrador	04/05/2026
Estado	MVP backend + UI EJS + admin + dataset curado

Indice

- 1. Resumen ejecutivo
- 2. Objetivos del proyecto
- 3. Alcance y decisiones de producto
- 4. Stack tecnologico
- 5. Arquitectura general
- 6. Modelo de datos
- 7. Diagramas tecnicos
- 8. API REST y endpoints
- 9. Interfaz EJS y experiencia de usuario
- 10. Autenticacion, sesiones y roles
- 11. Favoritos
- 12. Panel de administracion
- 13. Dataset, seed y datos de demo
- 14. Swagger/OpenAPI
- 15. Ejecucion local
- 16. QA y validaciones
- 17. Flujo Git y organizacion
- 18. Limitaciones
- 19. Mejoras futuras
- 20. Conclusion
- Anexos

1. Resumen ejecutivo

QuoteMatic es una aplicacion backend con minifront EJS orientada a recomendar frases segun la situacion del usuario, el tipo de frase y el rango de edad declarado. El proyecto se desarrolla como MVP de bootcamp Full Stack, con prioridad en demostrar dominio practico de Express, MongoDB, Mongoose, sesiones, autorizacion y API REST documentada.

El resultado final integra una API REST funcional, una interfaz EJS responsive para la demo, autenticacion con sesiones, roles user/admin, favoritos protegidos, panel de administracion para gestionar frases y autores, Swagger/OpenAPI y un dataset curado de demostracion con cobertura completa de filtros.

Area	Resultado
Backend	Express + TypeScript con rutas REST, controladores y modelos Mongoose.
Persistencia	MongoDB local mediante Docker Compose y modelos con indices relevantes.
Auth	Registro, login, logout, sesiones persistidas en MongoDB y roles.
UI	Vistas EJS para home, explorar frases, login/register, favoritos y admin.
Admin	CRUD de frases y autores protegido por rol admin.
Datos	Seed curado con 14 autores, 4 situaciones, 8 tipos y 156 frases.
Docs	README, documentos de sprint y Swagger en /api-docs.

2. Objetivos del proyecto

- Crear una aplicacion backend Full Stack sencilla, clonable y demostrable en local.
- Aplicar Node.js, Express, TypeScript, MongoDB y Mongoose con arquitectura por capas simple.
- Ofrecer una API REST clara para consultar frases, catalogos, favoritos y operaciones de admin.
- Incluir autenticacion real mediante sesiones y autorizacion por rol.
- Permitir una demo visual con EJS sin convertir el proyecto en un frontend complejo.
- Construir un dataset suficiente para que los filtros de la app no devuelvan resultados vacios.
- Documentar endpoints, flujo de ejecucion, decisiones tecnicas y limitaciones.

3. Alcance y decisiones de producto

La decision principal ha sido priorizar un MVP controlado.

La importacion desde APIs externas queda documentada como mejora futura, pero no forma parte del MVP actual.

Decision	Motivo
CRUD manual de frases y autores	Aporta control, trazabilidad y una demo estable.
Seed curado	Evita depender de APIs externas y garantiza cobertura de filtros.
EJS como minifront	Permite demo visual sin introducir React en el sprint backend.
Swagger publico	Facilita la correccion y prueba de endpoints.
Admin protegido	Las escrituras de frases y panel admin requieren rol admin.
Sin APIs externas activas	Se evita complejidad, caidas de red y datos no verificados.

4. Stack tecnologico

Tecnologia	Uso en el proyecto
Node.js	Runtime JavaScript para el servidor.
Express	Framework HTTP para rutas, middlewares, controladores y vistas.
TypeScript	Tipado estatico para modelos, controladores y scripts.
MongoDB	Base de datos NoSQL para usuarios, sesiones, frases y catalogos.
Mongoose	Modelado de colecciones, validaciones, referencias e indices.
EJS	Renderizado de vistas del minifront.
bcrypt	Hash de contrasenas.
express-session	Gestion de sesiones basadas en cookie.
connect-mongo	Persistencia de sesiones en MongoDB.
Swagger	Documentacion interactiva de la API REST.
Docker Compose	Servicio local de MongoDB reproducible.
tsx	Ejecucion TypeScript en desarrollo y scripts de seed.

5. Arquitectura general

La arquitectura sigue un patron pragmático cercano a MVC: las rutas reciben la peticion, los controladores gestionan la logica de aplicacion, los modelos Mongoose representan la persistencia y las vistas EJS se usan cuando la respuesta debe ser HTML.

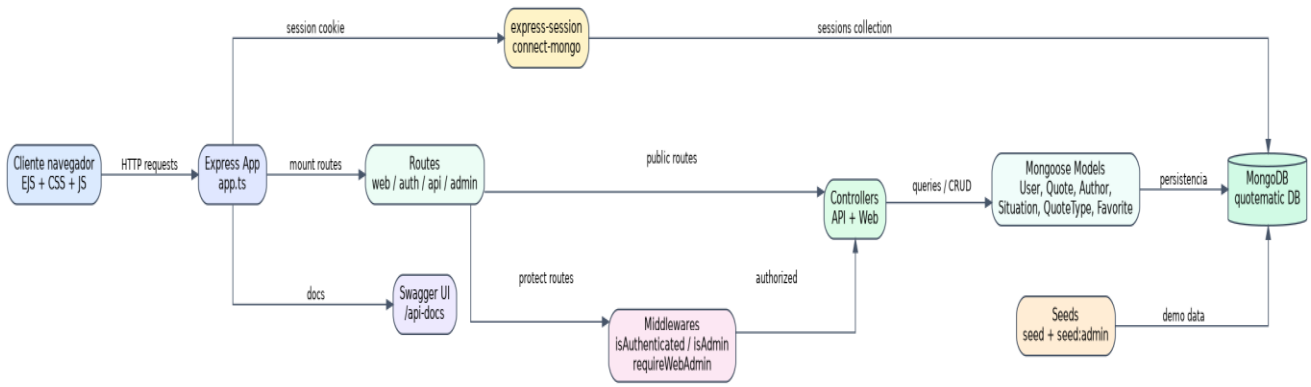

Figura 1. Arquitectura logica de QuoteMatic.

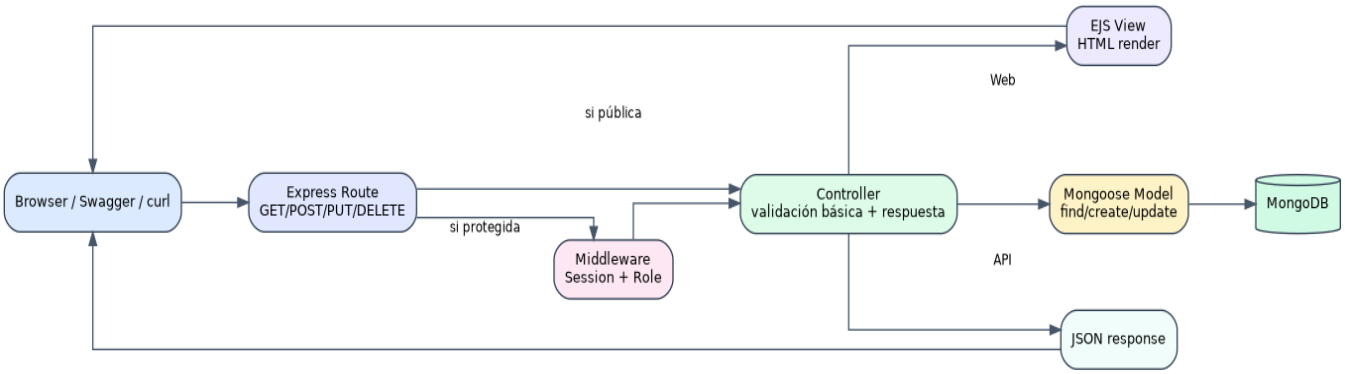

Figura 2. Flujo de una peticion web/API dentro del backend.

Capa	Responsabilidad
Routes	Definir URLs, metodos HTTP y middlewares aplicados.
Controllers	Procesar peticiones, validar datos basicos y llamar a modelos.
Middlewares	Control de sesion, autorizacion y acceso admin.
Models	Definir esquemas, enums, referencias e indices.
Views	Renderizar paginas EJS para la demo visual.
Seeds	Poblar datos de demo y usuario admin local.

6. Modelo de datos

El modelo de datos se compone de seis colecciones principales: User, Author, Situation, QuoteType, Quote y Favorite. Quote es la entidad central y referencia a Author, Situation y QuoteType.

Modelo	Descripcion	Indices o reglas destacadas
User	Usuario registrado con email, passwordHash, rol y ageGroup.	email unico; isActive indexado.
Author	Autor real, historico, ficticio o desconocido.	normalizedName unico.
Situation	Situacion de uso: trabajo, estudios, estres, decisiones dificiles.	slug unico.
QuoteType	Tipo de frase: stoic, philosophical, motivational, etc.	slug unico; contentRating asociado.
Quote	Frase con referencias a autor, situacion y tipo.	unique(textNormalized, author); isActive y verificationStatus indexados.
Favorite	Relacion usuario-frase para favoritos.	unique(user, quote); borrado logico con isActive.

7. Diagramas tecnicos

7.1 Diagrama ER

El diagrama ER resume las colecciones y las relaciones principales entre documentos. Aunque MongoDB no es relacional, Mongoose usa referencias ObjectId para conectar las entidades principales.

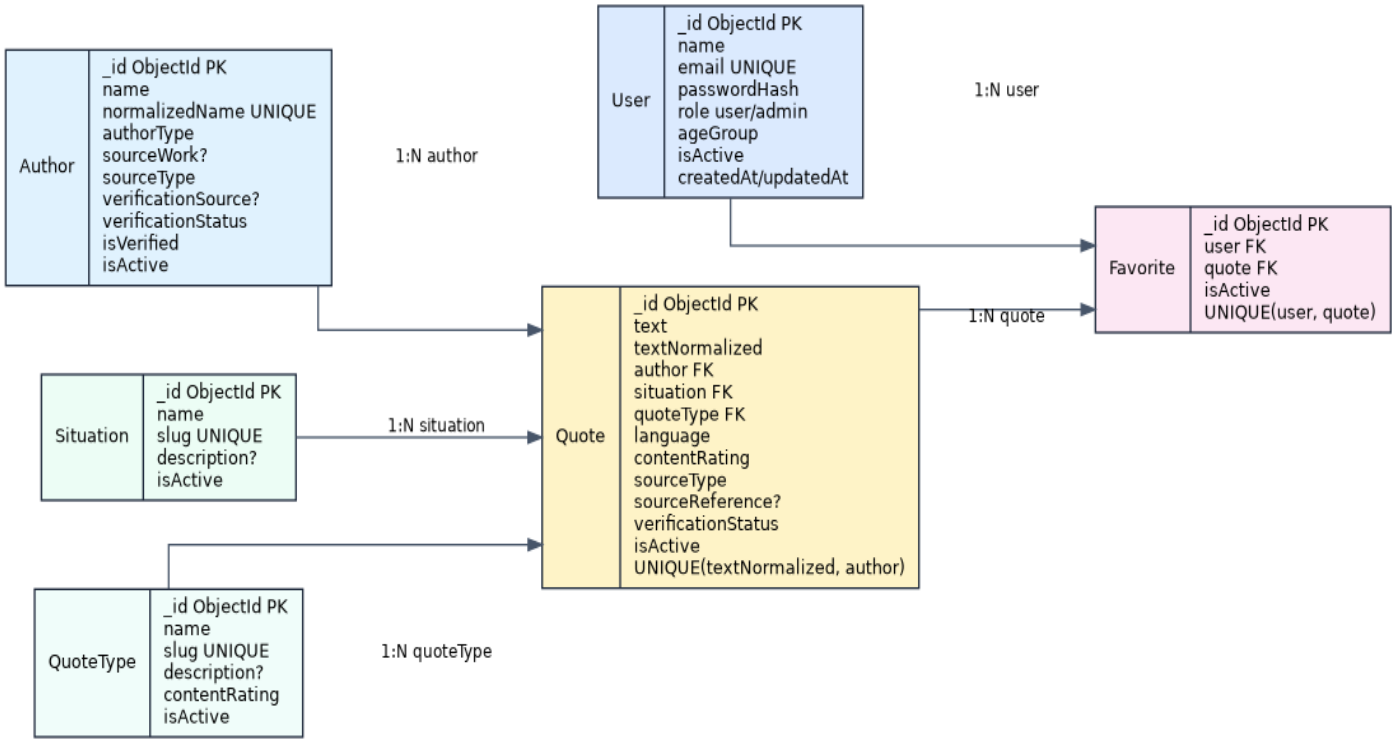

Figura 3. Diagrama ER de colecciones MongoDB/Mongoose.

7.2 Diagrama de clases/modelos

El diagrama de clases representa las interfaces TypeScript asociadas a los modelos Mongoose y las relaciones por referencia.

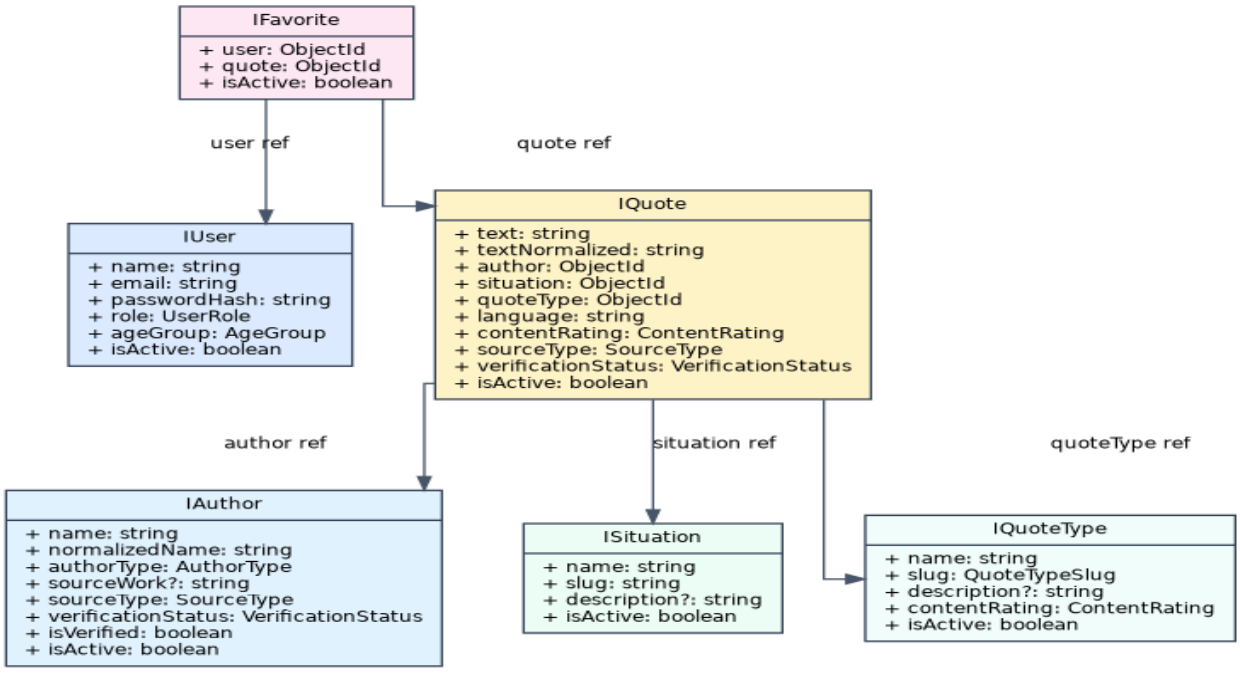

Figura 4. Diagrama de clases simplificado de los modelos.

8. API REST y endpoints

QuoteMatic expone endpoints publicos para consulta, endpoints protegidos por sesion para favoritos y endpoints protegidos por rol admin para escritura de frases.

Metodo	Ruta	Descripcion	Acceso
GET	/	Landing EJS publica	Publico
GET	/dashboard	Pantalla Explorar frases	Publico
GET	/favorites	Vista de favoritos del usuario	Sesion
GET	/admin	Panel admin	Admin
GET	/health	Health check	Publico
GET	/api-docs	Swagger UI	Publico
GET	/api-docs.json	OpenAPI JSON	Publico

Metodo	Ruta	Descripcion	Acceso
GET	/auth/register	Formulario de registro	Publico
POST	/auth/register	Crear usuario	Publico
GET	/auth/login	Formulario de login	Publico
POST	/auth/login	Iniciar sesion	Publico
POST	/auth/logout	Cerrar sesion	Sesion

GET	/auth/me	Comprobar sesion activa	Sesion
GET	/auth/admin-check	Comprobar rol admin	Admin

Metodo	Ruta	Descripcion	Acceso
GET	/api/quotes	Listar frases activas	Publico
GET	/api/quotes/random	Obtener frase aleatoria con filtros	Publico
GET	/api/quotes/:id	Obtener frase por id	Publico
POST	/api/quotes	Crear frase	Admin
PUT	/api/quotes/:id	Actualizar frase	Admin
DELETE	/api/quotes/:id	Borrado logico de frase	Admin

Metodo	Ruta	Descripcion	Acceso
GET	/api/authors	Catalogo de autores activos	Publico
GET	/api/situations	Catalogo de situaciones activas	Publico
GET	/api/quote-types	Catalogo de tipos de frase activos	Publico

Metodo	Ruta	Descripcion	Acceso
GET	/api/favorites/me	Listar favoritos activos del usuario	Sesion
POST	/api/favorites/:quoteId	Guardar frase como favorita	Sesion
DELETE	/api/favorites/:quoteId	Quitar frase de favoritos	Sesion

Metodo	Ruta	Descripcion	Acceso
GET	/admin/quotes	Listar frases desde admin	Admin
GET	/admin/quotes/new	Formulario nueva frase	Admin
POST	/admin/quotes	Crear frase con autor manual	Admin
GET	/admin/quotes/:id/edit	Editar frase	Admin
POST	/admin/quotes/:id/update	Actualizar frase	Admin
POST	/admin/quotes/:id/delete	Borrado logico desde admin	Admin
GET	/admin/authors	Listar autores	Admin
GET	/admin/authors/new	Formulario nuevo autor	Admin
POST	/admin/authors	Crear autor	Admin
GET	/admin/authors/:id/edit	Editar autor	Admin
POST	/admin/authors/:id/update	Actualizar autor	Admin
POST	/admin/authors/:id/delete	Borrado logico de autor	Admin

9. Interfaz EJS y experiencia de usuario

La interfaz EJS se disena como soporte visual del backend y no como frontend complejo. El objetivo es poder presentar la aplicacion desde navegador y demostrar los flujos principales sin depender solo de curl o Swagger.

Pantalla	Funcion
Home	Presenta QuoteMatic y ofrece un CTA claro hacia Explorar frases.
Explorar frases	Permite seleccionar situacion y tipo para pedir una frase aleatoria.
Login/Register	Gestiona acceso y creacion de usuario con formularios sencillos.
Favoritos	Muestra frases guardadas por el usuario autenticado.
Panel admin	Resume accesos a gestion de frases y autores.
Admin frases	CRUD visual de frases con autor escrito manualmente.
Admin autores	CRUD visual de autores.

10. Autenticacion, sesiones y roles

La autenticacion se basa en sesiones tradicionales. Las contrasenas se guardan con bcrypt y la sesion persiste en MongoDB mediante connect-mongo. Este enfoque es adecuado para una aplicacion EJS renderizada desde servidor.

Elemento	Detalle
Registro	Valida edad minima, normaliza email y crea usuario user activo.
Login	Comprueba email, bcrypt y usuario activo; guarda userId, role y ageGroup en sesion.
Logout	Destruye la sesion y redirige al inicio.
Rol user	Puede explorar frases y gestionar sus favoritos.
Rol admin	Accede al panel admin y a escritura de frases/autores.
Age gate	No permite registro de menores de 14 anos.

11. Favoritos

La funcionalidad de favoritos permite al usuario autenticado guardar frases y eliminarlas mediante borrado logico. Se evita duplicidad usando un indice unico compuesto por user y quote.

- GET /api/favorites/me devuelve los favoritos activos del usuario en sesion.
- POST /api/favorites/:quoteId crea o reactiva un favorito.
- DELETE /api/favorites/:quoteId marca el favorito como inactivo.
- Cada usuario solo ve y modifica sus propios favoritos.

12. Panel de administracion

El panel de administracion esta protegido por requireWebAdmin y se centra en dos entidades: frases y autores. La navegacion admin se simplifico para evitar elementos tecnicos o features fuera del MVP.

Gestion	Capacidades
Frases	Listar, crear, editar, actualizar y borrar logicamente.
Autores	Listar, crear, editar, actualizar y borrar logicamente.
Autor manual en frase	Al crear una frase, el admin escribe el nombre del autor. Si no existe, se crea automaticamente.
Subnavegacion	Solo muestra Frases y Autores.

13. Dataset, seed y datos de demo

El seed es una pieza central de la demo. Su objetivo no es representar una base editorial definitiva, sino ofrecer datos suficientes, coherentes y controlados para probar filtros, favoritos, roles y CRUD admin.

Recurso	Cantidad final
Autores	14
Situaciones	4
Tipos de frase	8
Frases	156
Combinaciones situation x quoteType	32/32 cubiertas
Minimo por combinacion	4 frases

13.1 Politica de atribucion

- Las frases atribuidas a autores reales se mantienen con verificationStatus pending si no hay validacion editorial completa.
- Las frases ficticias se atribuyen a personajes claramente identificables, como Yoda, Homer Simpson o Gandalf.
- Las frases originales de demo se atribuyen a QuoteMatic, Claude o Codex para no simular autoria real.
- Las frases anonimas se atribuyen a Anonimo.
- No hay importacion desde APIs externas en el MVP actual.

13.2 Seed destructivo

npm run seed borra y recrea datos de demo de Favorite, Quote, Author, Situation y QuoteType. No borra usuarios. El usuario admin de demo se gestiona por separado con npm run seed:admin.

```
docker compose up -d
npm run seed
npm run seed:admin
npm run dev
```

14. Swagger/OpenAPI

La documentacion Swagger esta disponible en /api-docs y la especificacion JSON en /api-docs.json. Sirve para probar endpoints y revisar la API sin depender de Postman.

- Documenta endpoints principales de health, auth, catalogos, quotes y favoritos.
- Los endpoints protegidos por sesion requieren cookie connect.sid en el navegador.
- POST/PUT/DELETE de quotes requieren rol admin.
- La documentacion es suficiente para demo academica, no una especificacion enterprise completa.

15. Ejecucion local

1. Instalar dependencias con npm install o npm ci.
2. Crear .env a partir de .env.example.
3. Levantar MongoDB con Docker Compose.
4. Ejecutar npm run seed para poblar datos de demo.
5. Ejecutar npm run seed:admin para crear el admin local.
6. Arrancar npm run dev y abrir http://localhost:3000.

```
PORT=3000
MONGODB_URI=mongodb://127.0.0.1:27017/quotematic
SESSION_SECRET=quotematic_dev_secret_change_me
```

Credencial demo	Valor
Email	admin@quotematic.local
Password	Admin123!
Rol	admin

16. QA y validaciones

La validacion se ha realizado de forma pragmatica mediante typecheck, build, seed, seed admin y pruebas manuales de rutas principales.

Validacion	Resultado esperado
npm run typecheck	TypeScript sin errores.
npm run build	Compila dist y copia vistas/assets.
npm run seed	Crea 14 autores, 4 situaciones, 8 tipos y 156 frases.
npm run seed:admin	Crea o actualiza admin@quotematic.local.
/	Home publica 200.
/dashboard	Explorar frases 200.
/favorites sin login	Redireccion a login.
/admin sin login	Redireccion a login.
/admin usuario normal	403 Forbidden.
/admin admin	200.
/api-docs/	Swagger 200.

17. Flujo Git y organizacion

El proyecto se ha trabajado por sprints y ramas de feature, con PR hacia dev. main queda como rama estable y dev como rama de integracion.

Rama/PR	Contenido
feat/favorites-api	Sprint 05: favoritos protegidos.
feat/admin-protected-quotes	Sprint 06: proteccion admin en escritura.
feat/swagger-openapi-docs	Sprint 07: Swagger/OpenAPI.
feat/ejs-responsive-ui	Sprint 08: UI EJS y admin.
chore/seed-demo-coverage	Dataset curado de demo.

18. Limitaciones

- No hay tests automatizados completos; la validacion principal es manual y por scripts.
- La interfaz EJS es funcional, pero no es un frontend SPA avanzado.
- Swagger esta disponible para facilitar la revision; no se ha protegido como area admin.
- No hay recuperacion de contrasena ni verificacion de email.
- El seed es destructivo y esta pensado para entorno local/demo.
- Las frases de demo no constituyen una base editorial verificada.

19. Mejoras futuras

- Importacion controlada desde APIs externas como Quotable, separada del MVP.
- Tests unitarios y de integracion para controllers, middlewares y modelos.
- Paginacion y busqueda avanzada de frases.
- Mejoras de seguridad para cookies en despliegue productivo.
- Frontend React en una fase posterior del bootcamp.
- Panel admin mas completo con para añadir bloques de frases en CSV.

20. Conclusion

QuoteMatic cumple el objetivo de proyecto backend de bootcamp: ofrece una API REST con MongoDB/Mongoose, autenticacion con sesiones, roles, favoritos, panel admin, Swagger y una UI EJS suficiente para una demo completa. La decision de mantener un MVP pequeno, sin depender de APIs externas, refuerza la estabilidad y facilita la evaluacion.

El dataset curado permite probar todas las combinaciones de filtros y reduce el riesgo de pantallas vacias durante la presentacion. El proyecto queda preparado para ser ampliado en el futuro con importacion externa, tests y un frontend React.

Anexo A. Resumen de scripts npm

Script	Descripcion
npm run dev	Servidor en modo desarrollo con tsx watch.
npm run typecheck	Validacion TypeScript sin emitir archivos.
npm run build	Limpia dist, compila TypeScript y copia assets.
npm start	Ejecuta dist/server.js.
npm run seed	Carga dataset de demo.
npm run seed:admin	Crea o actualiza usuario admin de demo.
npm run clean	Elimina dist.
npm run copy:assets	Copia vistas y public a dist.

Anexo B. Mapa rapido de rutas

Grupo	Rutas principales
Visual	/, /dashboard, /favorites, /admin
Auth	/auth/register, /auth/login, /auth/logout, /auth/me
Quotes API	/api/quotes, /api/quotes/random, /api/quotes/:id
Catalogos API	/api/authors, /api/situations, /api/quote-types
Favorites API	/api/favorites/me, /api/favorites/:quoteId
Admin web	/admin/quotes, /admin/authors
Docs	/api-docs, /api-docs.json